Offline-first PWA with a Service Worker

Problem: Build an offline-first notes app (think Flipkart Lite): it must open with no network, let the user keep adding and deleting notes offline, and sync those changes when the connection comes back. It must also tell the user when a new version is deployed.

Tech stack: React (Vite, JavaScript), plain Service Worker + Cache API, Web App Manifest

Time limit: 60 minutes

Often asked at: Flipkart (Flipkart Lite), Google. **Why it matters:** caching strategies.

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements** (functional + non-functional) and write down assumptions.
2. **List the interactions** (action -> expected result). This is your test checklist.
3. **Sketch the architecture**: modules, components, who talks to whom.
4. **Design the state**: what is stored, what is derived, who owns it.
5. **Define data shapes and contracts** (API payloads, mock data).
6. **Trace one key flow** end to end before coding.
7. **Name the hard parts** and decide how to solve them.
8. **Build in layers**: smallest working version first, then add one layer at a time, keeping it working.
9. **Cover states and edge cases**: loading, empty, error, offline, conflicts.
10. **Wrap up**: trade-offs, what you would do with more time.

Time budget for 60 minutes

Time	Work
0-7 min	Requirements, clarifying questions, assumptions
7-15 min	Architecture, state, contracts, caching strategy per request type
15-25 min	App online first: list + add + delete against the mock API
25-38 min	Service worker: precache shell, runtime strategies, offline reload
38-50 min	Outbox: offline writes, persistence, sync on reconnect
50-56 min	Update flow ("new version" banner), states and edge cases
56-60 min	Demo offline in DevTools, trade-offs

Think out loud

Say every decision and its reason as you make it: "*Navigations are network-first because a stale HTML page can point to bundles that no longer exist; static assets are hashed, so cache-first is safe.*" The interviewer grades your reasoning as much as your code. If you get stuck, say what you are trying, so they can nudge you.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- The app opens and works with **no network** after the first visit (app shell is cached).
- Shows the **last known list of notes** when offline.
- User can **add and delete notes offline**; changes appear instantly with a "Not synced" marker.
- Offline changes **survive a reload or closing the tab**.
- When back online, queued changes **sync automatically**, in order, without duplicates.
- Shows **Online / Offline** status and how many changes are waiting.
- When a new version is deployed, show **"A new version is available - Reload"**; never reload without asking.
- **Installable**: has a web app manifest and icon.

Non-functional requirements

Area	Requirement
Performance	Repeat visits load from cache (instant); slow networks ("lie-fi") fall back to cache after a timeout
Scalability	Caches are versioned and old versions are deleted; outbox is a small queue, not a full DB copy
Error handling	Network errors keep the change queued; server rejections (4xx) don't block the queue forever; storage-full is reported
Accessibility	Status in `role="status"` / `aria-live`, errors in `role="alert"`, labelled input, labelled delete buttons
Reusability	`useOnlineStatus`, outbox functions and the SW strategies are independent of the notes UI
Extensibility	New resource types = add an op type; new route = add a branch in the SW `fetch` handler

Clarifying questions to ask

1. Which data must work offline: read only, or also writes? (Assume both.)

2. If the same note is edited on two devices, who wins? (Assume no edits, only create/delete, so no merge conflicts.)
3. Is there a real backend, or can I mock it? (Assume a mock endpoint served by the dev server.)
4. Can I use Workbox, or should I write the service worker by hand? (Assume by hand to show the concepts; mention Workbox.)
5. Should the app auto-update or ask the user? (Assume ask, to never lose a half-typed note.)
6. Which browsers? Safari has no Background Sync. (Assume modern browsers, sync from the page.)

Assumptions we build with

- Mock API at `/api/notes` (GET list, POST create, DELETE by id), in memory, provided by a tiny Vite plugin.
- Only create and delete (no edit), so ordering the queue is enough; no merge logic.
- The service worker is registered only in the production build (`vite build && vite preview`).
- The outbox lives in `localStorage` (small, simple); IndexedDB is the upgrade path.
- Sync runs from the page on load and on the `online` event, not from the service worker.

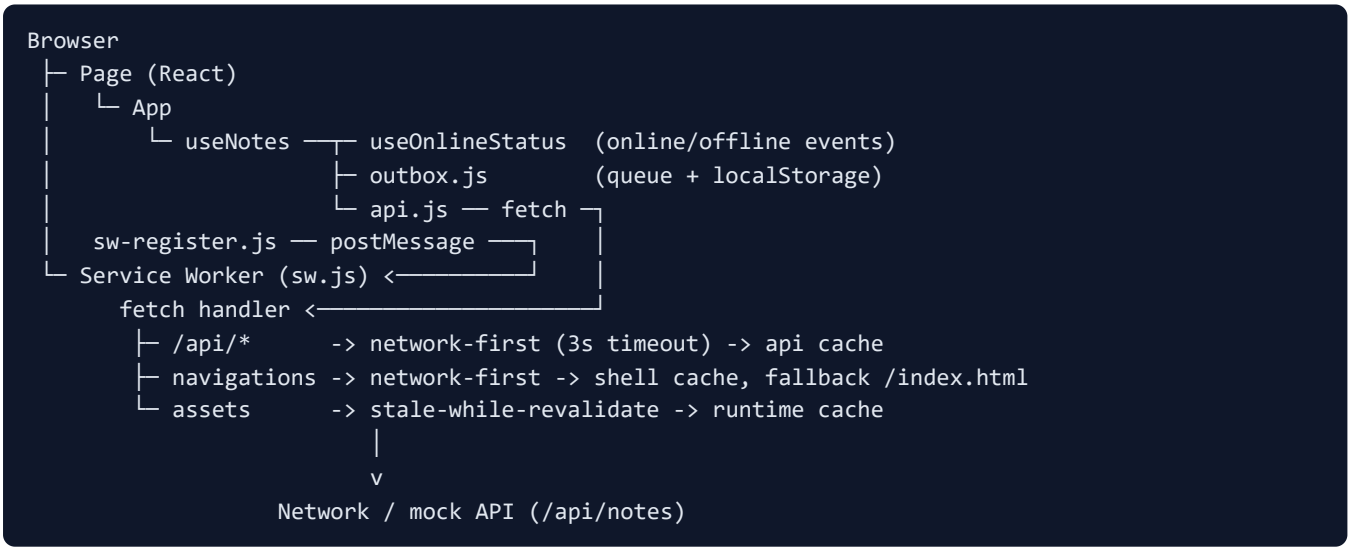
2. Identify interactions / use cases

Action / input	Expected behavior / output
First visit (online)	SW installs, precaches `index.html`, manifest, icon and the hashed JS/CSS
Reload while offline	App shell comes from cache; list comes from the cached `GET /api/notes`
Add a note (online)	Note appears at once, POST is sent, marker disappears when saved
Add a note (offline)	Note appears with "Not synced"; counter shows "1 change waiting to sync"
Delete a server note (offline)	Note disappears at once; delete is queued
Delete a note that was never synced	Its queued create is removed; nothing is sent to the server
Close tab offline, reopen	Queued changes and pending notes are still there
Connection comes back	Queue is sent oldest first; list refreshed; "All changes synced"
Server unreachable while "online"	Sync stops, changes stay queued, message "Sync paused...", "Sync now" button
Server rejects a change (400)	That change is dropped with an error message; the rest of the queue continues
Same POST sent twice (retry)	Server returns the same note (dedupe by `clientId`), no duplicate
New version deployed	Banner "A new version is available" with "Reload"
Click "Reload"	Waiting SW activates, old caches deleted, page reloads once

Action / input	Expected behavior / output
Empty list	"No notes yet. Add your first one."
Blank input	Add button disabled; whitespace-only text is ignored

3. Design the architecture

Module	Responsibility
`public/sw.js`	Service worker: precache app shell, route each GET to a caching strategy, clean old caches, activate on request
`src/sw-register.js`	Registers the SW, detects a waiting update, reloads only after the user agrees
`src/lib/api.js`	Thin `fetch` wrapper; throws `ApiError` with the HTTP status
`src/lib/outbox.js`	Pure queue logic (enqueue, dequeue, apply to list, classify errors) + persistence
`src/hooks/useOnlineStatus.js`	Subscribes to `online` / `offline` events
`src/hooks/useNotes.js`	Owns notes state, runs the sync loop, exposes `addNote`, `removeNote`, `sync`
`src/App.jsx`	UI: status badge, update banner, form, list
`vite.config.js`	Mock API plugin for dev and preview



Key split: the **service worker owns reads** (HTTP caching, transparent to the app), the **page owns writes** (the outbox). Start with only the page and the API; add the SW as a layer that the app does not need to know about.

4. Design the state / data model

State or field	Purpose
`serverNotes`	Last list the server (or the SW cache) returned
`outbox`	Writes not yet confirmed by the server, persisted in `localStorage`
`status`	`loading` (first fetch) / `idle` / `syncing`
`error`	Message for the user: sync paused, change rejected, storage full
`online`	From `navigator.onLine` + events (external store)
`applyUpdate` (App)	Function to activate a waiting SW, `null` when no update
`outboxRef`	Latest queue for the async sync loop (no stale closure)
`syncing` ref	Guard so two syncs never run at once

```
const online = useOnlineStatus()
const [serverNotes, setServerNotes] = useState([])
const [outbox, setOutbox] = useState(loadOutbox)       // lazy init from localStorage
const [status, setStatus] = useState('loading')        // loading | idle | syncing
const [error, setError] = useState(null)
const outboxRef = useRef(outbox)
const syncing = useRef(false)

// visible list is derived, never stored
const notes = useMemo(() => applyOutbox(serverNotes, outbox), [serverNotes, outbox])
```

Senior note:

- `notes` is **derived** = `serverNotes + pending creates - pending deletes`. Storing it separately would create two sources of truth that drift apart after a sync.
- `pendingCount` is `outbox.length`, derived.
- The **HTTP cache is owned by the service worker**, not by React state. The page does not keep its own copy of the list in `localStorage`; offline reads just go through `fetch` and the SW answers.
- The **outbox is owned by** `useNotes`. Every change goes through one `commit(next)` that updates the ref and the state together.

5. Design the data structures and contracts

Example data

```
// server note: id is created by the server
{ id: 's1', text: 'Welcome! Go offline and add a note.', createdAt: 1793637391371 }

// outbox ops: every op has its own opId; a create carries a client id
{ opId: '6f1c…', type: 'create', clientId: 'c-91ab…', text: 'Buy milk', createdAt: 1793637426962 }
{ opId: '0d2e…', type: 'delete', id: 's1' }
```

```
// pending note shown in the list (derived from a create op)
{ id: 'c-91ab…', text: 'Buy milk', createdAt: 1793637426962, pending: true }
```

Why stable ids: the `clientId` (from `crypto.randomUUID()`) is the note's key in the list while it is pending, lets a delete cancel a create that never reached the server, and is the **idempotency key** for the server: a retried POST with the same `clientId` returns the same note instead of a duplicate. `opId` identifies the op itself, so dequeue removes exactly the op that was sent.

API contracts (mock)

Request	Body	Response
`GET /api/notes`	-	`200 [{ id, text, createdAt }]`
`POST /api/notes`	`{ clientId, text, createdAt }`	`201 { id, text, createdAt }` (same note again for a repeated `clientId`); `400 { error }` if text or clientId missing
`DELETE /api/notes/:id`	-	`200 { id }`, also when the note is already gone (idempotent)
Any request, SW offline fallback	-	`503 { error: 'offline' }` when nothing is cached

Service worker contracts

Message / event	Direction	Meaning
`'SKIP_WAITING'`	page -> waiting SW	User clicked "Reload": activate now
`controllerchange`	browser -> page	New SW controls the page: reload (only if the user asked)
`sw-update` (CustomEvent)	`sw-register.js` -> App	`detail` is the function that applies the update

6. Trace the data flow

Add a note while offline, then reconnect:

```
User types "Buy milk" and presses Add
        |
        v
App.handleSubmit -> trim -> addNote('Buy milk')
        |
        v
useNotes.addNote: op = { opId, type:'create', clientId:'c-…', text, createdAt }
commit(enqueue(outboxRef.current, op))   -> outboxRef + state updated together
        |
        v
React re-renders: notes = applyOutbox(serverNotes, outbox)
-> "Buy milk  [Not synced]", "1 change waiting to sync"
```

```
        |
        v
effect: saveOutbox(outbox) -> localStorage (survives reload)
navigator.onLine is false -> no sync now
        |
        v
... network returns: 'online' event -> useOnlineStatus -> online = true
        |
        v
effect [online] -> sync()
  guard syncing.current; status = 'syncing'
  loop while outboxRef.current has ops:
     POST /api/notes {clientId, text, createdAt}
       -> SW ignores non-GET -> network -> 201 { id:'s9', ... }
     setServerNotes(prev => [...prev, saved])   (pending -> saved in one render)
     commit(dequeue(outboxRef.current, op.opId))
  setError(null); status = 'idle'
  refresh(): GET /api/notes -> SW network-first -> fresh list, cache updated
        |
        v
UI: "Buy milk" (no marker), "All changes synced"
```

7. Important technical challenges

Challenge 1: Choosing a caching strategy per request type

Problem: One strategy for everything breaks something. Cache-first for HTML: after a deploy the old `index.html` keeps loading and points to old bundles forever. Network-first for hashed JS: every load waits for the network for files that never change. Network-first with no timeout on a bad 2G connection: the app hangs for 30+ seconds instead of showing cached data.

Solution: Route by request type in the `fetch` handler.

Request	Strategy	Why
Navigations (HTML)	Network-first, 3s timeout, fallback cached `/index.html`	Fresh HTML when possible, works offline
Hashed assets (JS/CSS/icons)	Stale-while-revalidate	Instant from cache; a new hash is a new URL anyway
`GET /api/*`	Network-first, 3s timeout, fallback cache	Data should be fresh; cache is the offline backup
POST / DELETE	Not handled by SW	Cannot be cached; the app's outbox owns them

```
self.addEventListener('fetch', (event) => {
  const { request } = event
  const url = new URL(request.url)
  if (request.method !== 'GET' || url.origin !== self.location.origin) return
  if (url.pathname.startsWith('/api/')) event.respondWith(networkFirst(request, API_CACHE))
  else if (request.mode === 'navigate') event.respondWith(networkFirst(request, SHELL_CACHE,
'/index.html'))
```

```
  else event.respondWith(staleWhileRevalidate(request, RUNTIME_CACHE))
})

async function networkFirst(request, cacheName, fallbackUrl) {
  const cache = await caches.open(cacheName)
  try {
    const response = await Promise.race([fetch(request), timeout(3000)])
    if (response.ok) cache.put(request, response.clone()) // body can be read once: clone it
    return response
  } catch {
    const cached = (await cache.match(request)) || (fallbackUrl && (await
cache.match(fallbackUrl)))
    return cached || Response.json({ error: 'offline' }, { status: 503 })
  }
}
```

Challenge 2: The first offline reload must work (precaching hashed bundles)

Problem: On the first visit the page downloads `/assets/index-CWMnNHYg.js` **before** the SW is installed, so runtime caching never sees it. User goes offline and reloads: HTML comes from cache but the JS 404s, white screen. We also don't know the hashed names when writing `sw.js` by hand.

Solution: In `install`, cache the shell, then read the cached `index.html` and cache every `/assets/...` it links. `install` only succeeds if all of them are cached (`addAll` is atomic), so a half-cached version never activates. Old versions are deleted in `activate`.

```
async function precacheShell() {
  const cache = await caches.open(SHELL_CACHE)
  await cache.addAll(['/', '/index.html', '/manifest.webmanifest', '/icon.svg'])
  const html = await (await cache.match('/index.html')).text()
  const assets = [...html.matchAll(/(?:src|href)="(\/assets\/[^"]+)"/g)].map((m) => m[1])
  await cache.addAll(assets)
}

self.addEventListener('activate', (event) => {
  const keep = [SHELL_CACHE, RUNTIME_CACHE, API_CACHE]
  event.waitUntil(
    caches.keys()
      .then((keys) => Promise.all(keys.filter((k) => !keep.includes(k)).map((k) =>
caches.delete(k))))
      .then(() => self.clients.claim()),
  )
})
```

In a real project Workbox `injectManifest` generates the exact file list (including lazy chunks) at build time.

Challenge 3: Offline writes with an outbox that syncs safely

Problem: Three ways to lose or corrupt data:

1. User adds a note offline and closes the tab: an in-memory queue is gone.

2. The POST reached the server but the response was lost (connection dropped). We retry, and the server creates the note **twice**.
3. One change is invalid (server says 400). If we retry it forever, every later change is stuck behind it.

Also: user adds a note offline, then deletes it before syncing. Sending create + delete is wasteful; the server never needs to know.

Solution:

- Queue every write as an op in an **outbox persisted to** `localStorage`; the list is derived from `serverNotes + outbox`.
- **Idempotency key**: each create carries a `clientId`; the server returns the existing note for a repeated `clientId`.
- **Send oldest first, one at a time**; on network/5xx error stop and keep the op; on a permanent 4xx drop that op and continue.
- **Collapse ops**: a delete of a pending note just removes its create.

```
export function enqueue(outbox, op) {
  if (op.type === 'delete' && outbox.some((o) => o.type === 'create' && o.clientId === op.id)) {
    return outbox.filter((o) => o.clientId !== op.id) // never reached the server: cancel it
  }
  return [...outbox, op]
}

export const isPermanentError = (err) =>
  err.status >= 400 && err.status < 500 && ![408, 429].includes(err.status)

// in useNotes.sync()
while (outboxRef.current.length) {
  const op = outboxRef.current[0]
  try {
    const saved = await sendOp(op)
    /* swap pending -> saved note in serverNotes */
  } catch (err) {
    if (!isPermanentError(err)) throw err  // keep op, retry on next sync
    rejected = `A change was rejected by the server: ${err.message}`
  }
  commit(dequeue(outboxRef.current, op.opId))
}
```

The loop reads `outboxRef.current` (not the state from the closure), so a note added **while** a sync is running is picked up by the same loop. `commit()` updates the ref and state together. If the user deletes a note while its POST is in flight, the op is no longer in the queue when the response arrives, so we enqueue a delete for the new server id.

Challenge 4: Updating to a new version without breaking the open page

Problem: A new `sw.js` installs but **waits** while the old one controls open tabs. If we call `skipWaiting()` immediately, the new SW deletes the old caches while the old page is still running, and its next lazy chunk fails. If we reload automatically, the user loses the note they were typing. Also,

`clients.claim()` on the **first** install fires `controllerchange` too, so a naive "reload on controllerchange" reloads every first-time visitor.

Solution: Detect a waiting worker (only when a controller already exists = it's an update), show a banner, and on click send `SKIP_WAITING`. Reload on `controllerchange` only if the user asked.

```
let updateRequested = false
navigator.serviceWorker.addEventListener('controllerchange', () => {
  if (updateRequested) window.location.reload()
})

const reg = await navigator.serviceWorker.register('/sw.js')
if (reg.waiting && navigator.serviceWorker.controller) announce(reg.waiting)
reg.addEventListener('updatefound', () => {
  const worker = reg.installing
  worker.addEventListener('statechange', () => {
    if (worker.state === 'installed' && navigator.serviceWorker.controller) announce(worker)
  })
})
// announce() fires a CustomEvent whose detail sets updateRequested and posts 'SKIP_WAITING'
```

8. Implement in the correct order

#	Step	Working result
1	Vite app + mock API plugin (`GET/POST/DELETE /api/notes`)	`curl /api/notes` works
2	`api.js` + App: load list, add, delete (online only)	Basic notes app
3	`useOnlineStatus` + badge	Online/Offline shown
4	Manifest + icon + `<link rel="manifest">`	Installable
5	`sw.js` install: precache shell + hashed assets; register in prod	Offline reload shows the app
6	`fetch` routing: network-first for HTML/API, SWR for assets	Offline reload shows the last list
7	`activate` : delete old caches, `clients.claim()`	Versioned caches
8	`outbox.js` pure functions + self-check asserts	Queue logic tested
9	`useNotes` : all writes go through the outbox, derived `notes` , persist	Offline add/delete with "Not synced"
10	Sync loop: ordered, guarded, error classes, swap pending -> saved	Auto sync on reconnect
11	Update flow: banner + `SKIP_WAITING` + guarded reload	Safe deploys
12	States: loading, empty, errors, a11y roles, disabled button	Polished

9. Cover all states and edge cases

State / edge case	How it is handled
Initial (loading)	`status = 'loading'` -> "Loading notes..."
Success	List renders; "All changes synced"
Empty	"No notes yet. Add your first one."
Syncing	"Syncing...", "Sync now" hidden
Offline, data cached	SW returns cached `GET /api/notes`; red "Offline" badge
Offline, nothing cached (first ever visit offline)	Browser's offline page; nothing can be done before the first visit
Offline, API never cached	SW returns `503` -> "Could not load notes."; pending notes still visible
`navigator.onLine` true but server down	`fetch` throws -> "Sync paused...", ops kept, "Sync now" button
Slow network ("lie-fi")	3s timeout -> cached response
Server rejects op (400)	Op dropped, error shown, queue continues
429 / 5xx	Treated as temporary: kept and retried
Duplicate POST on retry	Server dedupes by `clientId`
Delete pending note	Create removed from queue; nothing sent
Delete pending note while its POST is in flight	Delete of the new server id is enqueued
Add while a sync is running	Same loop picks it up via `outboxRef`
Two syncs triggered together (online event + add)	`syncing` ref guard
Reload with pending changes	Outbox loaded from `localStorage`; synced on load if online
Corrupt `localStorage` value	`loadOutbox` catches JSON error -> empty queue
Storage full	`saveOutbox` throws -> "Storage is full..." warning
Whitespace-only / too long input	Trimmed + ignored; `maxLength=200`; server also validates
New version deployed	Banner; reload only on click
First install	`controllerchange` does not reload the page
SW unsupported / registration fails	App still works online; error logged
Dev mode	SW not registered, avoids stale modules

10. Final structure

```
offline-notes/
├─ index.html                # manifest link, theme-color
├─ vite.config.js            # react plugin + mock API (dev + preview)
├─ public/
│  ├─ sw.js                  # service worker (served at /sw.js, scope /)
│  ├─ manifest.webmanifest
│  └─ icon.svg
└─ src/
   ├─ main.jsx               # render + registerSW()
   ├─ sw-register.js         # registration + update detection
   ├─ App.jsx                # UI
   ├─ App.css
   ├─ lib/
   │  ├─ api.js              # fetch wrapper + ApiError
   │  └─ outbox.js           # pure queue logic + persistence
   └─ hooks/
      ├─ useOnlineStatus.js
      └─ useNotes.js         # state + sync loop
```

`sw.js` sits in `public/` so it is served from the root: a service worker can only control URLs at or below its own path.

11. Full working code

Run with `npm run build && npm run preview` (the service worker is only registered in the production build). Test offline in DevTools -> Application -> Service Workers -> "Offline", or Network -> "Offline".

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react-swc'

// Mock backend: in-memory notes at /api/notes, served by both `vite` and `vite preview`
function mockApi() {
  let notes = [{ id: 's1', text: 'Welcome! Go offline and add a note.', createdAt: Date.now() }]
  const byClientId = new Map() // clientId -> created note, so a retried POST never creates a
duplicate

  const send = (res, status, body) => {
    res.statusCode = status
    res.setHeader('Content-Type', 'application/json')
    res.end(JSON.stringify(body))
  }

  const handler = (req, res, next) => {
    if (!req.url.startsWith('/api/notes')) return next()

    if (req.method === 'GET') return send(res, 200, notes)

    if (req.method === 'DELETE') {
      const id = decodeURIComponent(req.url.split('/').pop())
```

```
      notes = notes.filter((n) => n.id !== id)
      return send(res, 200, { id }) // deleting a missing note is still a success (idempotent)
    }

    if (req.method === 'POST') {
      let raw = ''
      req.on('data', (chunk) => (raw += chunk))
      req.on('end', () => {
        let body
        try {
          body = JSON.parse(raw)
        } catch {
          return send(res, 400, { error: 'Invalid JSON' })
        }
        const text = String(body.text ?? '').trim()
        if (!text || !body.clientId) return send(res, 400, { error: 'text and clientId are
required' })
        if (!byClientId.has(body.clientId)) {
          const note = { id: `s${Date.now()}${Math.random().toString(36).slice(2, 6)}`, text,
createdAt: body.createdAt ?? Date.now() }
          byClientId.set(body.clientId, note)
          notes.push(note)
        }
        send(res, 201, byClientId.get(body.clientId))
      })
      return
    }

    send(res, 405, { error: 'Method not allowed' })
  }

  return {
    name: 'mock-api',
    // Braces matter: a returned function would be treated by Vite as a "post" middleware hook
    configureServer(server) {
      server.middlewares.use(handler)
    },
    configurePreviewServer(server) {
      server.middlewares.use(handler)
    },
  }
}

export default defineConfig({
  plugins: [react(), mockApi()],
})
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="theme-color" content="#2563eb" />
    <link rel="icon" href="/icon.svg" type="image/svg+xml" />
    <link rel="manifest" href="/manifest.webmanifest" />
```

```
    <title>Offline Notes</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

public/manifest.webmanifest

```
{
  "name": "Offline Notes",
  "short_name": "Notes",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#2563eb",
  "icons": [{ "src": "/icon.svg", "sizes": "any", "type": "image/svg+xml", "purpose": "any" }]
}
```

public/sw.js

```
// Bump on every deploy: a changed sw.js is what makes the browser install the new version
const VERSION = 'v1'
const SHELL_CACHE = `shell-${VERSION}`
const RUNTIME_CACHE = `runtime-${VERSION}`
const API_CACHE = `api-${VERSION}`
const SHELL_URLS = ['/', '/index.html', '/manifest.webmanifest', '/icon.svg']

self.addEventListener('install', (event) => {
  event.waitUntil(precacheShell())
  // No skipWaiting() here: the page asks for it when the user clicks "Reload"
})

self.addEventListener('activate', (event) => {
  const keep = [SHELL_CACHE, RUNTIME_CACHE, API_CACHE]
  event.waitUntil(
    caches
      .keys()
      .then((keys) => Promise.all(keys.filter((k) => !keep.includes(k)).map((k) =>
caches.delete(k))))
      .then(() => self.clients.claim()),
  )
})

self.addEventListener('message', (event) => {
  if (event.data === 'SKIP_WAITING') self.skipWaiting()
})

self.addEventListener('fetch', (event) => {
  const { request } = event
  const url = new URL(request.url)
  // POST/DELETE go straight to the network: the app's outbox handles them offline
  if (request.method !== 'GET' || url.origin !== self.location.origin) return
```

```
  if (url.pathname.startsWith('/api/')) event.respondWith(networkFirst(request, API_CACHE))
  else if (request.mode === 'navigate') event.respondWith(networkFirst(request, SHELL_CACHE,
'/index.html'))
  else event.respondWith(staleWhileRevalidate(request, RUNTIME_CACHE))
})

async function precacheShell() {
  const cache = await caches.open(SHELL_CACHE)
  await cache.addAll(SHELL_URLS)
  // The built index.html names the hashed bundles; cache them now so the very first offline
reload works.
  // shortcut: lazy chunks not linked from index.html are only runtime-cached, upgrade to Workbox
injectManifest for a full precache list
  const html = await (await cache.match('/index.html')).text()
  const assets = [...html.matchAll(/(?:src|href)="(\/assets\/[^"]+)"/g)].map((m) => m[1])
  await cache.addAll(assets)
}

const timeout = (ms) => new Promise((_, reject) => setTimeout(() => reject(new Error('timeout')),
ms))

// Fresh data when the network answers within 3s, cached copy otherwise (offline or "lie-fi")
async function networkFirst(request, cacheName, fallbackUrl) {
  const cache = await caches.open(cacheName)
  try {
    const response = await Promise.race([fetch(request), timeout(3000)])
    if (response.ok) cache.put(request, response.clone())
    return response
  } catch {
    const cached = (await cache.match(request)) || (fallbackUrl && (await
cache.match(fallbackUrl)))
    return cached || Response.json({ error: 'offline' }, { status: 503 })
  }
}

// Instant answer from cache, refresh the cache in the background for next time
async function staleWhileRevalidate(request, cacheName) {
  const cache = await caches.open(cacheName)
  const cached = await cache.match(request)
  const network = fetch(request)
    .then((response) => {
      if (response.ok) cache.put(request, response.clone())
      return response
    })
    .catch(() => cached || Response.error())
  return cached || network
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import { registerSW } from './sw-register.js'
import './App.css'
```

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

registerSW()
```

src/sw-register.js

```
export const SW_UPDATE_EVENT = 'sw-update'

// Registers /sw.js and fires SW_UPDATE_EVENT (detail = apply function) when a new version is
waiting
export function registerSW() {
  // Dev builds change on every save; a cache-first worker there would serve stale modules
  if (!('serviceWorker' in navigator) || !import.meta.env.PROD) return

  let updateRequested = false
  const announce = (worker) =>
    window.dispatchEvent(
      new CustomEvent(SW_UPDATE_EVENT, {
        detail: () => {
          updateRequested = true
          worker.postMessage('SKIP_WAITING')
        },
      }),
    )

  // The first install also fires controllerchange (clients.claim), so only reload after the user
asked
  navigator.serviceWorker.addEventListener('controllerchange', () => {
    if (updateRequested) window.location.reload()
  })

  window.addEventListener('load', async () => {
    try {
      const reg = await navigator.serviceWorker.register('/sw.js')
      if (reg.waiting && navigator.serviceWorker.controller) announce(reg.waiting)
      reg.addEventListener('updatefound', () => {
        const worker = reg.installing
        worker.addEventListener('statechange', () => {
          // A controller already exists, so this is an update, not the first install
          if (worker.state === 'installed' && navigator.serviceWorker.controller) announce(worker)
        })
      })
    } catch (err) {
      console.error('Service worker registration failed', err) // app still works online
    }
  })
}
```

src/lib/api.js

```
export class ApiError extends Error {
  constructor(status, message) {
    super(message)
    this.status = status
  }
}

async function request(url, options) {
  const res = await fetch(url, options) // throws TypeError when offline
  const body = await res.json().catch(() => ({}))
  if (!res.ok) throw new ApiError(res.status, body.error || `HTTP ${res.status}`)
  return body
}

export const fetchNotes = () => request('/api/notes')

export const createNote = ({ clientId, text, createdAt }) =>
  request('/api/notes', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ clientId, text, createdAt }),
  })

export const deleteNote = (id) => request(`/api/notes/${encodeURIComponent(id)}`, { method:
'DELETE' })
```

src/lib/outbox.js

```
// Outbox = queue of writes not yet confirmed by the server.
// op: { opId, type: 'create', clientId, text, createdAt } | { opId, type: 'delete', id }

export function enqueue(outbox, op) {
  // Deleting a note the server never saw: cancel its create instead of sending create + delete
  if (op.type === 'delete' && outbox.some((o) => o.type === 'create' && o.clientId === op.id)) {
    return outbox.filter((o) => o.clientId !== op.id)
  }
  return [...outbox, op]
}

export const dequeue = (outbox, opId) => outbox.filter((o) => o.opId !== opId)

// What the user sees = server list + pending creates - pending deletes
export function applyOutbox(serverNotes, outbox) {
  const deleted = new Set(outbox.filter((o) => o.type === 'delete').map((o) => o.id))
  const pending = outbox
    .filter((o) => o.type === 'create')
    .map((o) => ({ id: o.clientId, text: o.text, createdAt: o.createdAt, pending: true }))
  return [...serverNotes.filter((n) => !deleted.has(n.id)), ...pending]
}

// 4xx (except timeout / rate limit) will fail the same way forever: drop it instead of blocking
the queue
export const isPermanentError = (err) => err.status >= 400 && err.status < 500 && ![408,
429].includes(err.status)
```

```
// shortcut: localStorage is sync and page-only, move to IndexedDB if the service worker must
replay the queue (Background Sync)
const KEY = 'notes-outbox'

export function loadOutbox() {
  try {
    return JSON.parse(localStorage.getItem(KEY)) ?? []
  } catch {
    return []
  }
}

export const saveOutbox = (outbox) => localStorage.setItem(KEY, JSON.stringify(outbox))
```

src/hooks/useOnlineStatus.js

```
import { useSyncExternalStore } from 'react'

function subscribe(callback) {
  window.addEventListener('online', callback)
  window.addEventListener('offline', callback)
  return () => {
    window.removeEventListener('online', callback)
    window.removeEventListener('offline', callback)
  }
}

// navigator.onLine only means "has a network", not "server reachable": failed requests are
handled separately
export const useOnlineStatus = () => useSyncExternalStore(subscribe, () => navigator.onLine)
```

src/hooks/useNotes.js

```
import { useCallback, useEffect, useMemo, useRef, useState } from 'react'
import { createNote, deleteNote, fetchNotes } from '../lib/api.js'
import { applyOutbox, dequeue, enqueue, isPermanentError, loadOutbox, saveOutbox } from
'../lib/outbox.js'
import { useOnlineStatus } from './useOnlineStatus.js'

const sendOp = (op) => (op.type === 'create' ? createNote(op) : deleteNote(op.id))

export function useNotes() {
  const online = useOnlineStatus()
  const [serverNotes, setServerNotes] = useState([])
  const [outbox, setOutbox] = useState(loadOutbox)
  const [status, setStatus] = useState('loading') // loading | idle | syncing
  const [error, setError] = useState(null)

  // The sync loop needs the latest queue without waiting for a re-render
  const outboxRef = useRef(outbox)
  const syncing = useRef(false)

  const commit = useCallback((next) => {
    outboxRef.current = next
```

```
    setOutbox(next)
  }, [])

  useEffect(() => {
    try {
      saveOutbox(outbox)
    } catch {
      setError('Storage is full: offline changes will be lost if you close the app.')
    }
  }, [outbox])

  const refresh = useCallback(async () => {
    try {
      setServerNotes(await fetchNotes()) // offline: the service worker answers from its cache
    } catch {
      setError('Could not load notes.')
    } finally {
      setStatus((s) => (s === 'loading' ? 'idle' : s))
    }
  }, [])

  // Send queued writes one by one, oldest first, so the server sees them in the order the user
made them
  const sync = useCallback(async () => {
    if (syncing.current) return
    syncing.current = true
    setStatus('syncing')
    let rejected = null
    try {
      while (outboxRef.current.length) {
        const op = outboxRef.current[0]
        try {
          const saved = await sendOp(op)
          const cancelled = !outboxRef.current.some((o) => o.opId === op.opId)
          if (op.type === 'create' && cancelled) {
            // User deleted the note while its POST was in flight: delete the server copy too
            commit(enqueue(outboxRef.current, { opId: crypto.randomUUID(), type: 'delete', id:
saved.id }))
          } else if (op.type === 'create') {
            // Swap pending -> saved in the same render, so the note never blinks out before
refresh()
            setServerNotes((prev) => [...prev.filter((n) => n.id !== saved.id), saved])
          } else {
            setServerNotes((prev) => prev.filter((n) => n.id !== op.id))
          }
        } catch (err) {
          if (!isPermanentError(err)) throw err // network/5xx: keep the op, retry on the next
sync
          rejected = `A change was rejected by the server: ${err.message}`
        }
        commit(dequeue(outboxRef.current, op.opId))
      }
      setError(rejected)
    } catch {
      setError('Sync paused: server unreachable. Changes are saved on this device.')
    } finally {
      syncing.current = false
      setStatus('idle')
      await refresh()
```

```
    }
  }, [commit, refresh])

  // Mount + every reconnect: flush the queue when online, read the cached list when offline
  useEffect(() => {
    if (online) sync()
    else refresh()
  }, [online, sync, refresh])

  const addNote = useCallback(
    (text) => {
      const op = { opId: crypto.randomUUID(), type: 'create', clientId:
`c-${crypto.randomUUID()}`, text, createdAt: Date.now() }
      commit(enqueue(outboxRef.current, op))
      if (navigator.onLine) sync()
    },
    [commit, sync],
  )

  const removeNote = useCallback(
    (id) => {
      commit(enqueue(outboxRef.current, { opId: crypto.randomUUID(), type: 'delete', id }))
      if (navigator.onLine) sync()
    },
    [commit, sync],
  )

  const notes = useMemo(() => applyOutbox(serverNotes, outbox), [serverNotes, outbox])

  return { notes, online, status, error, pendingCount: outbox.length, addNote, removeNote, sync }
}
```

src/App.jsx

```
import { useEffect, useState } from 'react'
import { useNotes } from './hooks/useNotes.js'
import { SW_UPDATE_EVENT } from './sw-register.js'

const MAX_LENGTH = 200

export default function App() {
  const { notes, online, status, error, pendingCount, addNote, removeNote, sync } = useNotes()
  const [text, setText] = useState('')
  const [applyUpdate, setApplyUpdate] = useState(null)

  useEffect(() => {
    const onUpdate = (e) => setApplyUpdate(() => e.detail)
    window.addEventListener(SW_UPDATE_EVENT, onUpdate)
    return () => window.removeEventListener(SW_UPDATE_EVENT, onUpdate)
  }, [])

  const handleSubmit = (e) => {
    e.preventDefault()
    const value = text.trim()
    if (!value) return
    addNote(value)
    setText('')
```

```
  }

  return (
    <main className="app">
      <header className="header">
        <h1>Offline Notes</h1>
        <span className={`badge ${online ? 'online' : 'offline'}`} role="status">
          {online ? 'Online' : 'Offline'}
        </span>
      </header>

      {applyUpdate && (
        <div className="banner" role="alert">
          A new version is available.
          <button onClick={applyUpdate}>Reload</button>
        </div>
      )}

      <p className="sync" aria-live="polite">
        {status === 'syncing'
          ? 'Syncing…'
          : pendingCount
            ? `${pendingCount} change${pendingCount > 1 ? 's' : ''} waiting to sync`
            : 'All changes synced'}
        {pendingCount > 0 && online && status !== 'syncing' && <button onClick={sync}>Sync
now</button>}
      </p>

      {error && (
        <p className="error" role="alert">
          {error}
        </p>
      )}

      <form className="form" onSubmit={handleSubmit}>
        <label htmlFor="note" className="sr-only">
          New note
        </label>
        <input
          id="note"
          value={text}
          onChange={(e) => setText(e.target.value)}
          maxLength={MAX_LENGTH}
          placeholder="Write a note (works offline)"
        />
        <button type="submit" disabled={!text.trim()}>
          Add
        </button>
      </form>

      {status === 'loading' ? (
        <p className="muted">Loading notes…</p>
      ) : notes.length === 0 ? (
        <p className="muted">No notes yet. Add your first one.</p>
      ) : (
        <ul className="list">
          {notes.map((note) => (
            <li key={note.id} className={note.pending ? 'pending' : ''}>
              <span>{note.text}</span>
```

```
              {note.pending && <small className="tag">Not synced</small>}
              <button onClick={() => removeNote(note.id)} aria-label={`Delete note:
${note.text}`}>
                ×
              </button>
            </li>
          ))}
        </ul>
      )}
    </main>
  )
}
```

src/App.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f4f6fb; color: #111827; }
button { cursor: pointer; font: inherit; }

.app { max-width: 560px; margin: 0 auto; padding: 24px 16px; }
.header { display: flex; align-items: center; justify-content: space-between; }
.header h1 { margin: 0; font-size: 1.5rem; }

.badge { padding: 4px 10px; border-radius: 999px; font-size: 0.8rem; font-weight: 600; }
.badge.online { background: #dcfce7; color: #166534; }
.badge.offline { background: #fee2e2; color: #991b1b; }

.banner { display: flex; justify-content: space-between; align-items: center; gap: 8px; margin-
top: 16px; padding: 10px 12px; border-radius: 8px; background: #dbeafe; }
.sync { display: flex; align-items: center; gap: 8px; color: #4b5563; font-size: 0.9rem; }
.error { padding: 8px 12px; border-radius: 8px; background: #fef2f2; color: #b91c1c; }
.muted { color: #6b7280; text-align: center; }

.form { display: flex; gap: 8px; margin: 16px 0; }
.form input { flex: 1; padding: 10px 12px; border: 1px solid #d1d5db; border-radius: 8px; font:
inherit; }
.form button { padding: 10px 16px; border: 0; border-radius: 8px; background: #2563eb; color:
#fff; }
.form button:disabled { opacity: 0.5; cursor: not-allowed; }

.list { list-style: none; margin: 0; padding: 0; display: grid; gap: 8px; }
.list li { display: flex; align-items: center; gap: 8px; padding: 12px; border-radius: 8px;
background: #fff; border: 1px solid #e5e7eb; }
.list li span { flex: 1; word-break: break-word; }
.list li.pending { border-style: dashed; }
.list li button { border: 0; background: none; color: #9ca3af; }
.list li button:hover { color: #dc2626; }
.tag { font-size: 0.75rem; color: #b45309; }

.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); }
```

selfcheck.mjs (run with `node selfcheck.mjs`)

```
import assert from 'node:assert/strict'
import { applyOutbox, dequeue, enqueue, isPermanentError } from './src/lib/outbox.js'
```

```
const server = [{ id: 's1', text: 'a', createdAt: 1 }]
const create = { opId: 'o1', type: 'create', clientId: 'c1', text: 'b', createdAt: 2 }

let q = enqueue([], create)
assert.deepEqual(applyOutbox(server, q).map((n) => n.id), ['s1', 'c1'])
assert.equal(applyOutbox(server, q)[1].pending, true)

// delete of an unsynced note cancels its create
assert.deepEqual(enqueue(q, { opId: 'o2', type: 'delete', id: 'c1' }), [])

// delete of a server note hides it until synced
q = enqueue(q, { opId: 'o3', type: 'delete', id: 's1' })
assert.deepEqual(applyOutbox(server, q).map((n) => n.id), ['c1'])
assert.deepEqual(dequeue(q, 'o1').map((o) => o.opId), ['o3'])

assert.equal(isPermanentError({ status: 400 }), true)
assert.equal(isPermanentError({ status: 429 }), false)
assert.equal(isPermanentError({ status: 503 }), false)
assert.equal(isPermanentError(new TypeError('Failed to fetch')), false)
console.log('selfcheck ok')
```

12. Wrap-up

Edge cases handled

- First offline reload works (hashed bundles precached at install).
- Lie-fi: 3s timeout falls back to cache.
- Offline writes survive reload; ordered, idempotent, de-duplicated sync.
- Poison changes (4xx) don't block the queue; temporary failures (5xx, 429, network) retry.
- Delete-before-sync cancels the create; delete-during-sync deletes the server copy.
- Concurrent sync triggers guarded; changes during a sync are picked up.
- Update never reloads without consent; first install never reloads; old caches removed.

Trade-offs to mention

Decision	Trade-off
Hand-written SW instead of Workbox	Shows understanding; Workbox gives precache manifests, expiration, Background Sync for free
Outbox in `localStorage`	Simple and synchronous, but ~5 MB, blocks the main thread, and the SW cannot read it (no Background Sync)
Sync from the page	Works in all browsers including Safari; but only while a tab is open
Network-first for API	Fresh data online, but each online load waits for the network (up to 3s)
Asking before update	Safe for the user, but some users stay on the old version until they click
No conflict handling	Fine for create/delete only; edits would need versions or last-write-wins

Decision	Trade-off
Parsing `index.html` for assets	Zero build config; misses lazy chunks not linked from the HTML

What to add with more time

- **IndexedDB** for the outbox and notes (via `idb`), plus **Background Sync** (`sync` event) so queued changes send even after the tab is closed (Chrome).
- **Workbox** `injectManifest` for an exact precache list, cache expiration (max entries / max age) for runtime and API caches.
- Exponential backoff with jitter for retries; a retry counter per op.
- Edit support with conflict resolution (version numbers, `If-Match` / ETag, or last-write-wins with server timestamps).
- "Last synced at" timestamp and a "showing cached data" hint when a response came from cache.
- Tests: unit tests for `outbox.js` (already asserts), SW strategies with `msw` or a fake `caches`, Playwright e2e with `context.setOffline(true)`.
- Lighthouse PWA audit, maskable PNG icons (192/512), offline analytics queue.

---

13. Counter questions the interviewer may ask

Requirements

Q1. What exactly makes an app a "PWA"?

A web app served over HTTPS (localhost is allowed) with a **web app manifest** (name, icons, `start_url`, `display`) and a **service worker** that makes it work offline. Together they make it installable and reliable. Everything else (push, background sync) is optional.

Q2. What does "offline-first" mean compared to "offline fallback"?

Offline fallback shows an "you're offline" page when the network fails. Offline-first treats the network as an enhancement: the app always renders from local data first, writes go to a local queue first, and the network syncs in the background. Our app does reads via the SW cache and writes via the outbox, so the user can keep working.

Q3. Why not register the service worker in development?

Vite serves modules that change on every save. A stale-while-revalidate worker would serve the previous version of a module and you'd debug ghosts. So we register only when `import.meta.env.PROD`, and test with `vite build && vite preview`. Workbox's Vite plugin does the same by default.

State design

Q4. Why derive the visible list instead of storing it?

`notes = serverNotes + pending creates - pending deletes`. If we stored a separate `notes` array, every sync step would have to update two places, and one missed update leaves a phantom or missing note. Deriving it with `useMemo` makes that bug impossible.

Q5. Why is there a `useRef` for the outbox when you already have state?

The sync loop is async and runs across many renders. Reading `outbox` from its closure would give the queue as it was when `sync` was created, so a note added mid-sync would be missed. `outboxRef.current` is always the latest; `commit()` writes ref and state together so they never disagree.

Q6. Why not keep the notes list in `localStorage` too?

The SW already caches `GET /api/notes`; a second copy in `localStorage` is a second source of truth to keep in sync. Keeping reads in the HTTP cache and only writes in the outbox gives each store one job. With IndexedDB for everything, you'd drop the API cache instead.

Challenge 1: caching strategies

Q7. Explain cache-first, network-first and stale-while-revalidate. When is each wrong?

- **Cache-first**: fastest; wrong for HTML or data, which go stale forever.
- **Network-first**: freshest; wrong for hashed assets (slow for no reason), and without a timeout it hangs on bad networks.
- **Stale-while-revalidate**: instant and refreshes in the background; wrong when the user must see the latest data on this load (prices, stock). Also: network-only (POST) and cache-only (precached shell).

Q8. Why `response.clone()` before `cache.put`?

A `Response` body is a stream and can only be read once. We need it twice: once for the cache, once for the page. Without `clone()` one of them gets "body already used".

Q9. What if the API is slow, not down?

That's "lie-fi": `navigator.onLine` is true but requests take 20 seconds. `Promise.race([fetch(request), timeout(3000)])` gives up after 3 s and serves the cache. The fetch keeps going in the background; with a little more code you could still write its result into the cache when it arrives.

Q10. How do you keep caches from growing forever?

Version the cache names and delete every unknown cache in `activate`. For runtime caches, add expiration: max entries or max age (Workbox `ExpirationPlugin`), and respect storage pressure (`navigator.storage.estimate()`, `navigator.storage.persist()`).

Challenge 2: precaching and lifecycle

Q11. Walk me through the service worker lifecycle.

`register` -> download -> **install** (precache; `waitUntil` fails the install if any file fails) -> **waiting** (if an older SW controls open tabs) -> **activate** (clean old caches, `clients.claim()`) -> controls fetches. A new version is detected when the browser sees a byte-different `sw.js`; it checks on navigations (and you can force it with `reg.update()`, e.g. on an interval).

Q12. If `sw.js` doesn't change but the app's bundles do, what happens?

The browser sees no update, so no new install. Navigations are network-first, so the new `index.html` and its new bundle URLs still load online, and SWR caches them. But the precache still holds the old bundles. That's why `VERSION` must change on every deploy; in a real build you inject a build hash into `sw.js` (Workbox does this via the precache manifest revision).

Q13. Why not `skipWaiting()` + `clients.claim()` right away and skip the banner?

It's fine for a first install, but for an update the new SW would start serving a page that was built for the old one: the old page may lazy-load a chunk whose hash only exists in the old cache that we just deleted. Asking the user and reloading keeps page and SW on the same version.

Q14. What is the scope of a service worker and why is `sw.js` in `public/`?

A SW controls URLs at or below the path it's served from. `/assets/sw.js` could only control `/assets/*`. Putting it in `public/` serves it at `/sw.js`, scope `/`. Never give `sw.js` a hashed name: the browser looks for updates at the same URL.

Challenge 3: outbox and sync

Q15. What if the POST succeeded but the response never arrived?

The op stays in the outbox and is sent again later. Because it carries the same `clientId`, the server returns the already-created note instead of making a duplicate. That's an **idempotency key**; real APIs often take it as an `Idempotency-Key` header.

Q16. Why send ops one at a time instead of `Promise.all`?

Order matters: create then delete of the same note must reach the server in that order. Sequential also makes "stop at the first network failure" simple. For large queues, batch them into one request (`POST /api/notes/batch`) rather than parallelising.

Q17. What if two devices edit the same note offline?

We avoided this by supporting only create/delete. With edits: send the version you based your edit on (`If-Match: <etag>`); the server returns `409` on mismatch, and you either apply last-write-wins, show a conflict UI, or use a CRDT for text. Pick based on how bad a lost edit is.

Q18. Why not use Background Sync?

It lets the SW replay the queue even after the tab is closed, which is better. But it's Chromium-only (no Safari/Firefox), and the SW can't read `localStorage`, so the queue would have to move to IndexedDB. Page-based sync works everywhere; Background Sync is a progressive enhancement on top.

Q19. Can you trust `navigator.onLine`?

Only when it says **false**. `true` means "connected to some network", not "server reachable" (captive Wi-Fi, server down). So we use it only to trigger a sync attempt, and treat a thrown `fetch` as the real signal: keep the op and show "Sync paused".

Challenge 4: updates

Q20. Why does the first install not reload the page?

`clients.claim()` makes the new SW take control of the already-open page, which fires `controllerchange`. If we reloaded on every `controllerchange`, every new visitor would see a surprise reload. The `updateRequested` flag means we only reload after the user clicked "Reload".

Performance and scale

Q21. What if the user has 1 million notes?

Don't fetch the whole list: paginate the API and cache pages, keep notes in IndexedDB with an index on `createdAt`, render with a virtualised list, and sync by delta (`GET /api/notes?since=<cursor>`) instead of refetching everything after each sync.

Q22. What if thousands of changes are queued offline?

`localStorage` writes the whole array on every change (O(n) and synchronous), so move the queue to IndexedDB (one record per op). Collapse ops per note before sending, and send in batches with backoff so reconnect doesn't hammer the server.

Trade-offs / "why not X?"

Q23. Why not just use Workbox?

In production I would: it generates the precache list, handles revisioning, expiration and Background Sync. In the interview, writing it by hand shows I understand install/activate/fetch and each strategy, and Workbox is then just a nicer API for the same ideas.

Q24. Why not use HTTP caching (`Cache-Control`) instead of a service worker?

HTTP caching speeds up online loads but cannot serve anything when the network request fails, cannot give a fallback page, and can't queue writes. Use both: long `max-age, immutable` for hashed assets, `no-cache` for `index.html` and `sw.js`, and the SW for offline.